

论文解构报告

Revisiting Pipeline Parallelism for LLM Serving

Soonjae Hwang, Jeongseob Ahn

20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26) · 2026

pipeline parallelism

LLM serving

chunked prefill

tensor parallelism

SGLang

目录

摘要翻译	3
1. 一句话总览	3
2. 阅读范围与证据状态	3
3. 根问题：论文究竟要解决什么	3
4. 旧方法为何不够	4
5. 核心洞见与假设	5
6. 方法：从洞见到可执行系统	5
7. 为什么它可能有效	6
8. 实验与指标：证据是否支撑主张	6
9. 图表解读与论证关系	7
10. 完整因果推理树	10
11. 结论、贡献与边界	11
12. 术语与第一性原理学习路径	11

摘要翻译

由于单个 GPU 的显存容量不足以容纳大语言模型（LLM），模型并行已成为在多 GPU 上服务 LLM 的标准方法。在在线服务环境中，张量并行凭借其并行执行降低计算延迟的能力，已成为单节点多 GPU 系统中事实上的标准做法。尽管流水线并行能够提供更高的吞吐量，但它存在流水线不均衡的问题，且该问题在在线负载下会被加剧，导致资源利用率下降和性能退化。在本研究中，我们重新审视了用于 LLM 服务的流水线并行。我们的分析表明，流水线各阶段之间的计算不均衡在在线服务场景下会进一步加剧。为解决这些流水线低效问题，我们提出了三种技术：贪心和预测两种机制，用于动态调整分块（chunk）大小以缓解由预填充（prefill）引发的气泡；以及一种延迟调度技术，用于动态重新平衡各流水线阶段的解码负载，从而进一步减少流水线气泡。我们在 SGLang 之上实现了这些技术，并证明在四块 NVIDIA A100 40GB GPU 上运行 Qwen2.5 32B 和 14B 模型时，采用我们机制的流水线并行性能优于张量并行。

1. 一句话总览

论文元数据

字段	内容
标题	Revisiting Pipeline Parallelism for LLM Serving
作者	Soonjae Hwang, Jeongseob Ahn
发表场所 / 年份	20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26), 2026
研究领域	系统（LLM 推理服务）
关键词	pipeline parallelism, LLM serving, chunked prefill, tensor parallelism, SGLang

资源链接

- 原始文件：本地上传文件
- arXiv：文中未说明
- DOI：文中未说明
- 代码 / 数据集：文中未说明

标签：流水线并行 · LLM 在线推理服务 · 动态分块调度 · 延迟调度 · SGLang 系统实现

论文重新审视流水线并行（PP）在 LLM 在线服务中的可行性，指出其性能瓶颈是在线负载下各阶段计算不均衡引发的“流水线气泡”，并提出贪心/预测式动态分块与延迟调度三项技术，在 4×A100（PCIe）上使 PP 的 goodput 超过张量并行（TP）[作者明确陈述](#)。

2. 阅读范围与证据状态

本次分析覆盖第 2-5 节正文（约第 2-15 页）及 6 组图表证据（共约 12 张图，另有 2 张因超出上限未做视觉分析）。文本层面结论多可核实到页码；部分超参数取值、抢占阈值窗口、MAPE 等指标公式、与相关工作 [6]、phase-disaggregated serving 的直接对比实验均“文中未说明”或“无法从当前 OCR 内容确认”。图像部分存在坐标轴标签重叠（如 ShareGPT TPOT/E2E 子图）、多曲线高度重合导致无法逐点读数等真实缺口，均已在对应小节注明，不影响整体因果链的可靠性。

3. 根问题：论文究竟要解决什么

问题定义：在单机多 GPU、仅有 PCIe（无 NVLink）互联的在线服务环境下，能否让流水线并行在满足 TTFT/TPOT 的 SLO（服务承诺延迟上限）前提下取得比张量并行更高的吞吐（goodput，即满足 SLO 前提下能达到的最大请求速率）[作者明确陈述](#)。

为什么重要：单 GPU 显存无法容纳大模型全部参数，模型并行是刚需；TP 依赖高带宽互联隐藏频繁的全局归约通信，但许多商用 GPU 服务器只有 PCIe，通信开销成为瓶颈 [作者明确陈述](#)。PP 通信量小、天然适合带宽受限部署，若能解决其气泡问题，就能为大量非 NVLink 服务器提供“低通信 + 高吞吐”方案 [基于文中逻辑的推断](#)。

困难来源：PP 的核心新约束是“流水线气泡”——阶段间计算量不均衡导致部分 GPU 空闲等待，且在线服务下请求到达时间与输入长度的不可预测变化使这种不均衡比训练场景更严重 [作者明确陈述](#)。

本文的 story：旧路线在“静态”这一技术特征上失效——现有 PP 实现的静态微批调度和 chunked-prefill 的静态分块大小都无法响应负载变化；本文的转折是用两类动态机制（分块大小、请求重排）替代静态策略，从而在不改变通信模式的前提下压平计算不均衡。

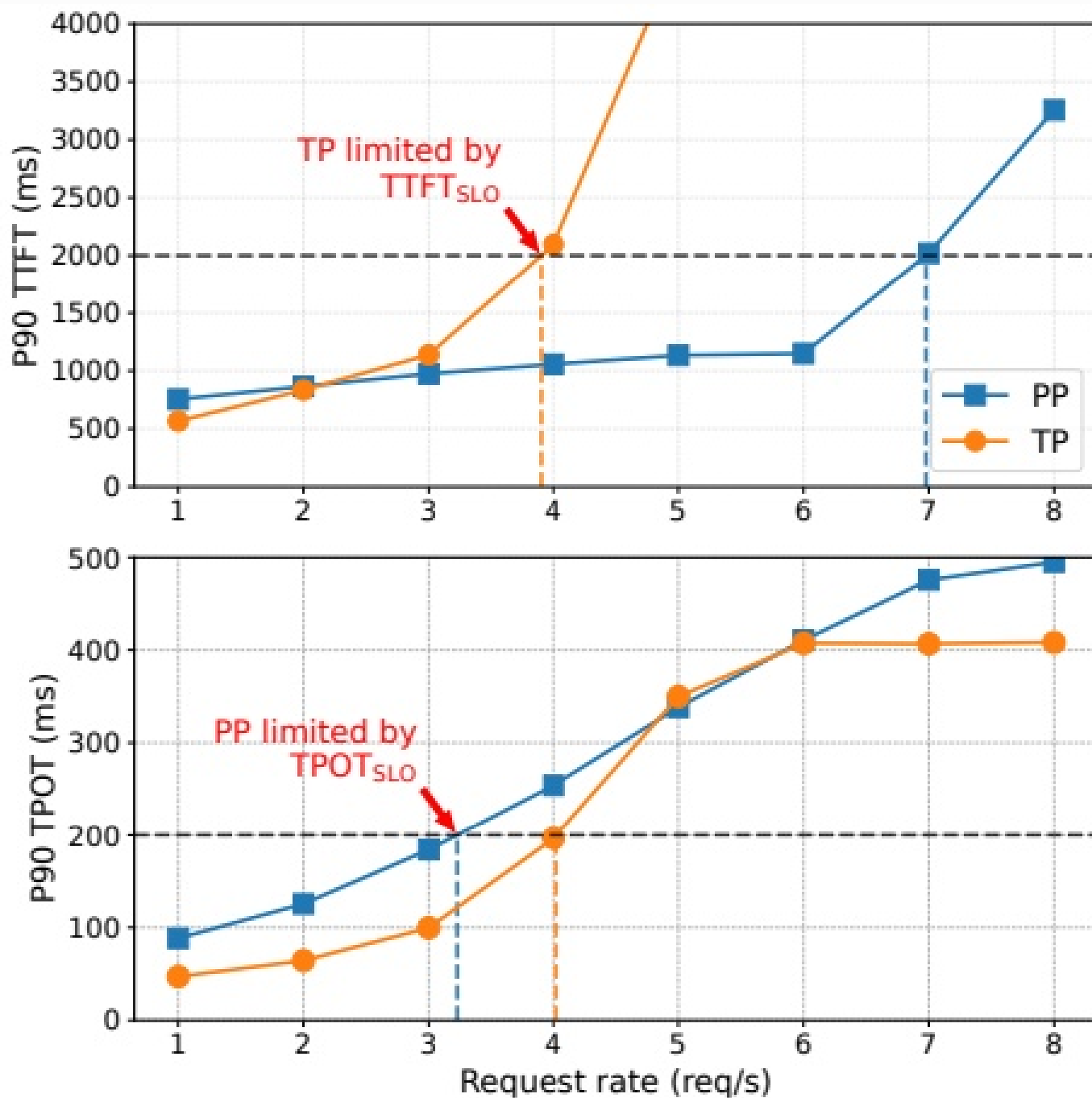


Figure 1: Performance comparison for pipeline and tensor parallelism in serving Qwen2.5-32B with the Azure Conversation workload on 4 NVIDIA A100 GPUs

图组解读

- **图组结论**：请求率 1-6 区间 PP 的 TTFT 优于 TP，但 TPOT 在 3.2 处先于 TP（4 处）违反 SLO，TP goodput 边界为 PP 的 1.2 倍。
- **论证关系**：为根问题（PP 是否劣于 TP）提供量化起点，证实流水线气泡是主要瓶颈。
- **适用范围**：覆盖 4×A100 PCIe 部署，不解释气泡微观成因。

4. 旧方法为何不够

张量并行 (TP)

前提：解决多卡显存不足下的负载均衡（每 GPU 执行相同操作，均衡是“trivial”的）

本文指出的失效点

依赖高带宽互联隐藏通信；在 PCIe 环境下通信开销成为瓶颈，不适合在线服务 **作者明确陈述**

本文对应模块

流水线并行 (PP) 结合动态机制 (DCP+DS) 压平计算不均衡

检验: Figure 1 展示的 TP vs PP 在 SLO 达标下的请求率差异 **实验支持**

Chunked-prefill (Sarathi-Serve)

前提: 解决长 prefill 独占迭代导致 decode 延迟飙升的问题

本文指出的失效点

分块大小静态设定——块大气泡多、块小吞吐低, 需要权衡而未做动态调整 **作者明确陈述**

→

本文对应模块

贪心/预测式动态分块 (DCP_greedy / DCP_pred)

检验: Figure 4 (气泡/吞吐 vs chunk size)、5.2.1 节 PP_static vs DCP 对比 **实验支持**

现有 PP 静态微批调度 (vLLM、SGLang)

前提: 实现简单 (请求固定分配到微批)

本文指出的失效点

在 decode-heavy 场景下放大不平衡、反复造成停顿 **作者明确陈述**

→

本文对应模块

延迟调度 (DS)

检验: CNN-Reversed 场景下 DS vs PP/DCP 的 goodput 对比 **实验支持**

Phase-disaggregated serving

前提: 减少 prefill/decode 阶段干扰

本文指出的失效点

引入模型权重复制与高带宽 KV cache 传输开销 **作者明确陈述**

→

本文对应模块

在不复制权重、不传输 KV cache 的前提下利用动态机制压平计算不均匀

检验: 本文未与之做直接实验对比 [无法从当前 OCR 内容确认]

近期动态分块方法 [6]

前提: 关注 SLO 合规

本文指出的失效点

忽视流水线并行的结构性约束, 未解决气泡问题 **作者明确陈述**

→

本文对应模块

综合考虑流水线并行结构约束与气泡调控的预测式动态分块 (DCP_pred)

检验: 无直接数值对比 [无法从当前 OCR 内容确认]

5. 核心洞见与假设

核心洞见: 流水线气泡的根本原因是阶段间计算不平衡, 且这种不平衡在在线服务中比训练更严重, 因为请求到达与输入长度不可预测 **作者明确陈述**。作者将其拆解为三类: P-P (prefill 长度差异)、P-D (prefill 与 decode 计算量悬殊)、D-D (纯 decode 时请求数分配不均, 且线性层存在“阶梯函数”式效率骤降) **作者明确陈述**。

假设 (待实验验证): 只要动态调整 chunk size 平衡 prefill 引起的不平衡, 并动态在微批间迁移 decode 请求平衡请求数, 即可显著减少气泡, 使 PP 吞吐超过 TP **[作者明确陈述, 待实验验证]**。该假设成立的前提是不平衡主要来自可调度层面 (chunk 大小、请求分配), 而非模型结构本身; 论文未讨论多租户或 MoE 场景下前提是否仍成立 [无法从当前 OCR 内容确认]。

6. 方法: 从洞见到可执行系统

总览: 每次调度迭代读取系统状态 (KV 空闲比例、延迟余量、队列/批次信息), 依次经过内存安全检查、chunk size 决策 (贪心或预测)、必要时的跨阶段请求重排, 输出该迭代实际执行的微批配置。

- 模块一: 贪心式动态分块 (DCP_greedy)** ——动机: 静态 chunk 无法适应负载变化, 需要改动小、易落地的反馈方案 **作者明确陈述**。输入 $TTFT_{slack}$ 、 $TPOT_{slack}$ 、 KV_{free} ; 先做内存安全检查, 再依据两个 slack 值增大或减小 chunk size **作者明确陈述**。优势: 实现简单; 缺口: 反应式反馈存在滞后, 需多轮收敛 **作者明确陈述**。
- 模块二: 预测式动态分块 (DCP_pred / ALP)** ——动机: 克服贪心法只能被动响应的结构性限制 **作者明确陈述**。核心是延迟预测模型:

$$T_{iter} = T_{offline} + T_{online}$$

其中

$$T_{offline} = \alpha_0 N_{tokens} + \alpha_1$$

描述与 chunk size 成正比的线性层延迟,

$$T_{online} = \beta_0 \sum (L_{past} L_{pre}) + \beta_1 \sum (L_{pre}^2) + \beta_2 \sum L_{dec} + \varepsilon$$

描述随上下文变化的 attention 延迟,

β

系数用 RLS（在线最小二乘）实时校准 [作者明确陈述](#)。对每个候选 chunk 预测延迟，剔除超 $TPOT_{SLO}$ 的候选得到上界，按 $TTFT_{SLO}$ 所需最小吞吐得到下界，选满足两界的最小候选（无候选时退化为满足 $TPOT_{SLO}$ 的最大 chunk）[作者明确陈述](#)。

3. **模块三：延迟调度（DS）**——动机：动态分块只对 prefill 引起的不均衡有效，对纯 decode 的 D-D 不均衡无效 [作者明确陈述](#)。当某微批请求数超出硬件高效批次边界（如 128 的倍数）时，依据 $TPOT_{slack}$ 决定抢占老请求还是新请求，临时挂起超额请求并在资源释放后按序恢复 [作者明确陈述](#)。

三模块串联：DCP 处理 prefill 相关气泡，DS 处理纯 decode 气泡，二者互补而非替代。

前排论文大图 [方法流水线](#) 呼应：第 6 节：模块二预测式动态分块（DCP_pred）

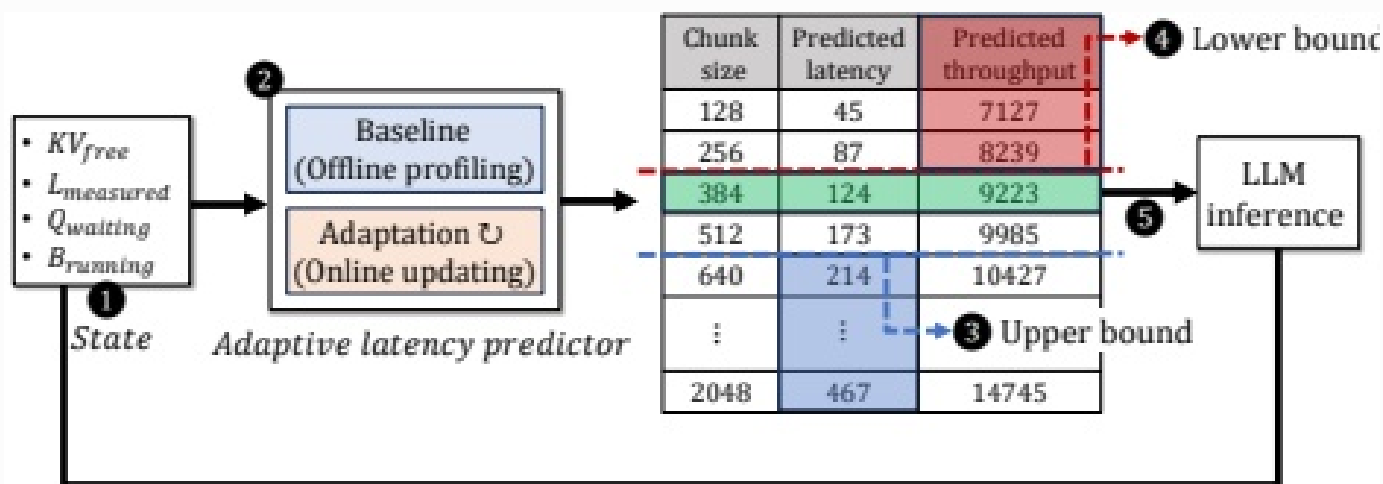


Figure 6: Workflow of predictive dynamic chunked-prefill

图组解读

- **图组结论**：展示结合离线基线与在线 RLS 校准、按上下界约束筛选候选 chunk 的五步闭环决策流程。
- **论证关系**：解释预测式动态分块的在线推理机制，衔接算法与预测公式。
- **适用范围**：决策流程示意，图中具体数值非可核验实测结果。

7. 为什么它可能有效

- 动态 chunk 让 linear 层延迟（与 chunk size 成正比）随负载伸缩，从而在低负载减小气泡、高负载保吞吐 [基于文中逻辑的推断](#)。
- RLS 在线校准使预测模型对量纲差异大的特征（如 L_{pre}^2 与 L_{dec} ）不敏感，并能快速适应 prefill-decode 干扰的变化，这是预测式优于贪心式的关键前提 [作者明确陈述](#)。
- DS 通过挂起-恢复机制让微批请求数对齐硬件高效批次边界，规避线性层的“阶梯函数”效率骤降 [基于文中逻辑的推断](#)，其有效性的前提是不均衡确实来自请求数而非 chunk size，即仅在 decode-heavy 场景下才需要。

8. 实验与指标：证据是否支撑主张

8.1 实验问题与判定标准

- 效果问题：调优后的 PP（含 DCP/DS）能否在 goodput/TTFT/TPOT 上超过 TP？对应 5.2.1、5.2.5 节的直接对比 [实验支持](#)。
- 机制问题：气泡是否确由 P-P/P-D/D-D 三类不均衡驱动？对应 3.1.1/3.1.2 节及 Figure 2/3 [实验支持](#)。
- 必要性问题：DCP 与 DS 是否各自不可替代？对应 CNN-Reversed 场景下 DCP-only vs DCP+DS 对比 [实验支持](#)。
- 鲁棒性/适用范围问题：优势在不同模型规模（32B/14B）、不同 SLO 严格程度下是否稳健？对应 5.2.3 节变 SLO 实验 [实验支持](#)。

8.2 数据、任务、对比与运行设置

- 硬件：4×NVIDIA A100 40GB，PCIe 4.0 互联 [作者明确陈述](#)。
- 模型：Qwen2.5 32B、14B [作者明确陈述](#)。

- 负载: AzureConv、ShareGPT、CNN-DailyMail (均为 prefill-heavy)、CNN-Reversed (人工构造的 decode-heavy 对照) **作者明确陈述**。
- 基线: TP、PP_static (多个固定 chunk: 128/256/512/1024/2048) **作者明确陈述**。
- SLO: TTFT $\leq$ 2000ms, TPOT $\leq$ 200ms (MLPerf 基准), 5.2.3 节另设 SLO scale 从 1.0 降至 0.2 的敏感性梯度 **作者明确陈述**。
- 重复次数/统计口径: 文中未说明。
- 关键超参数 ($\theta_{mem}, \alpha, \beta, \gamma, \delta$ 等取值) 及抢占阈值窗口: 文中未说明 [无法从当前 OCR 内容确认]。

8.3 指标字典与对应公式

- **指标与方向**: Goodput, 单位 req/s, 越高越好。**定义与公式**: 作者文字定义——“同时满足 P90 TTFT 与 P90 TPOT SLO 的最大请求速率” [原文公式 (文字定义)]。**实验用途**: 核心评估指标, 比较 TP/PP/DCP/DS。**读数与边界**: 见 8.4; 不能单独说明气泡的具体成因。
- **指标与方向**: $T_{offline}$, 线性层迭代延迟 (ms), 描述性。**定义与公式**: $T_{offline} = \alpha_0 N_{tokens} + \alpha_1$, 原文公式 (无 Eq. 编号); N_{tokens} 即 chunk size, α_0, α_1 为离线拟合系数。**实验用途**: 预测模型延迟估计的基础项。**读数与边界**: 仅为估计公式, 未给出具体系数取值。
- **指标与方向**: T_{online} , attention 层迭代延迟 (ms), 描述性。**定义与公式**: $T_{online} = \beta_0 \sum (L_{past} L_{pre}) + \beta_1 \sum (L_{pre}^2) + \beta_2 \sum L_{dec} + \varepsilon$, 原文公式 (无编号); 变量含义见第 6 节。**实验用途**: 与 $T_{offline}$ 相加得 T_{iter} , 驱动预测式分块决策。**读数与边界**: β 系数由 RLS 在线更新, 静态取值未给出。
- **指标与方向**: MAPE, 预测精度误差 (%), 越低越好。**定义与公式**: 无可核验公式, 论文仅提及“测量 MAPE”。**实验用途**: 比较 ALP 消融版本与决策树/DNN 的预测精度 (Table 1)。**读数与边界**: 完整 ALP=3.26%, 仅线性层建模=15.85%, 决策树/DNN 略优 (2.74%/2.56%) 但需离线训练 **实验支持**。
- **指标与方向**: TTFT/TPOT (ms), 越低越好。**定义与公式**: 无可核验公式, 仅文字描述为“评估 prefill/decode 阶段性性能的延迟指标”。**实验用途**: SLO 达标判定与主结果折线图纵轴。**读数与边界**: 见 8.4 具体数值。

8.4 主结果、消融与证据边界 比较工作与被批判限制的呼应

比较工作/基线	核心限制或失效条件	本文对应模块	直接实验与仍未验证的边界
TP	PCIe 下通信开销瓶颈	DCP+DS	Figure 1/9/10/12/13 数值对比; 未测试 NVLink 混合场景
静态 chunked-prefill	块大小固定, 气泡-吞吐权衡未解决	DCP_greedy/DCP_pred	Figure 4、5.2.1 节; 未与 [6] 直接对比
静态微批调度	decode-heavy 场景放大不均衡	Delay Scheduling	CNN-Reversed 场景各项对比; 抢占阈值未公开

主张与证据强度

主张	直接证据	证据强度与边界
调优后 PP 在 goodput/延迟上超过 TP	Figure 1/9/10, 5.2.1/5.2.5 节数值	实验支持, 覆盖两个模型规模四条负载
DCP_pred 优于 DCP_greedy 尤其在尾延迟	Figure 11 CDF, P90/P99 数值	实验支持, 但两曲线常高度重合难以像素级区分
DS 仅在 decode-heavy 下有效	5.2.1/5.2.3 节, CNN-Reversed 组图	实验支持; 14B 模型上 DS 对 P90 E2E 无可见改善

- AzureConv 上 DCP_greedy/DCP_pred 相对 PP_{static} 降低 TPOT/E2E 19%/18%、35%/31%; CNN 上 27%/28%、42%/36% **实验支持**。
- SLO scale 从 1.0 降至 0.4 时, DCP 相对 PP 的 goodput 提升从 $2.7\times$ 增至 $5.4\times$ (32B); SLO scale=0.2 时 32B 所有 PP 系统 goodput 归零 **实验支持**。
- decode-heavy 合成负载下 DS 相较 DCP_pred 平均降低 E2E 4062ms, 但 P99 TPOT 变差 15% (抢占代价) **实验支持**。
- 真实 trace (5.2.5 节): TP 仅 1.9% SLO attainment, PP 默认配置 1.6%, PP_{static} (离线搜索) 达 92.5%; 严格 TPOT SLO 下 DCP_greedy/DCP_pred 再提升 $3.06\times/4.14\times$ **实验支持**。
- 与 phase-disaggregated serving、方法 [6] 均无直接数值对比实验 [无法从当前 OCR 内容确认]。

9. 图表解读与论证关系

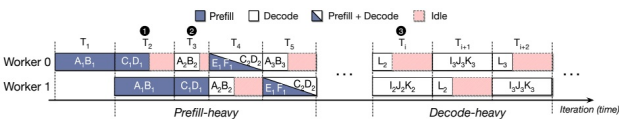


Figure 2: Three types of pipeline bubbles (The subscript indicates the iteration number.)

图组解读 **消融验证** 呼应: 第 5 节: 三类计算不均衡核心洞见

- **图组结论**: 甘特图展示 P-P (不同长度 prefill)、P-D (prefill 与 decode 混流)、D-D (decode 请求数不均) 三种触发空闲的场景。
- **论证关系**: 为 DCP (处理 P-P/P-D) 与 DS (处理 D-D) 的模块划分提供机制依据。
- **适用范围**: 简化教学示意, 非实测数据, 不提供量化时长。

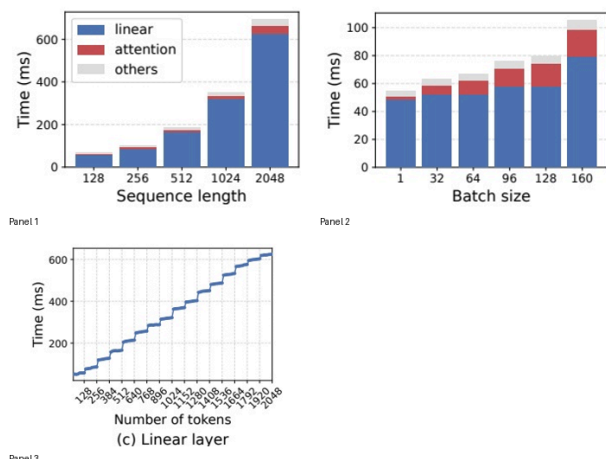


Figure 3: Performance characteristics: (a) prefill time when varying the sequence length, (b) decode time when increasing the batch size, and (c) linear operations when increasing the number of tokens

图组解读 消融验证 呼应：第 5 节：不均衡的性能量化与物理约束

- **图组结论**: prefill 延迟随序列长度近似线性增长且以 linear 层为主; decode batch size 增大时 linear 层呈阶梯状效率骤降。
- **论证关系**: 为 $T_{offline} = \alpha_0 N_{tokens} + \alpha_1$ 及 DS 中“对齐 128 倍数”的设计提供经验依据。
- **适用范围**: 仅能确认阶梯模式, 无法精确读出特定相邻点跳变值。

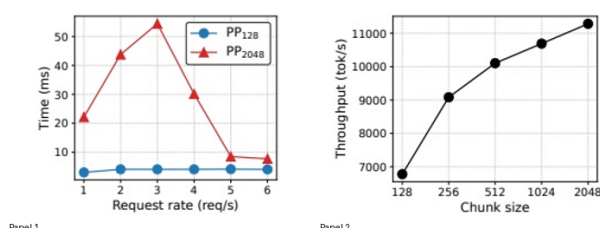


Figure 4: Pipeline bubbles and prefill token throughput according to load and chunk size (when serving Qwen2.5-32B with the Azure Code workload on 4 NVIDIA A100 GPUs)

图组解读 对比基线 呼应：第 4 节：静态 Chunked-prefill 缺陷

- **图组结论**: 大 chunk (2048) 在低负载下气泡显著高于小 chunk (128), 但吞吐随 chunk 增大单调递增且边际递减。
- **论证关系**: 直接证明静态 chunk 无法兼顾低气泡与高吞吐, 是 DCP 模块存在的必要性证据。
- **适用范围**: 展示静态基线权衡困境, 不直接证明动态方法效果。

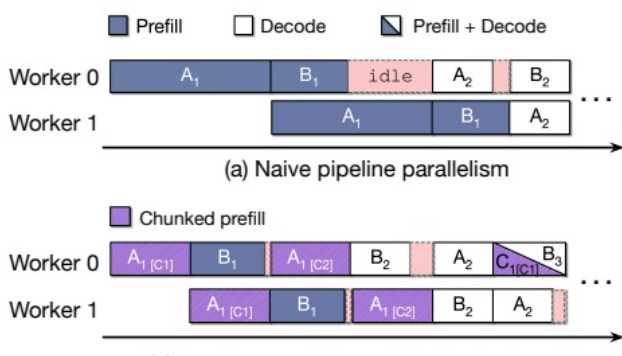


Figure 5: Reducing pipeline stalls from chunked-prefill [2]

图组解读 对比基线 呼应：第 6 节：Chunked-prefill 基础机制

- **图组结论**: 切分长 prefill 为多个小 chunk 可允许流水线前段提前推进, 消除特定的空闲等待。
- **论证关系**: 解释 chunked-prefill 减少气泡的基础机制, 衔接动态 chunk size 设计动机。
- **适用范围**: 两工位流水线简化示意, 非真实系统运行 trace。

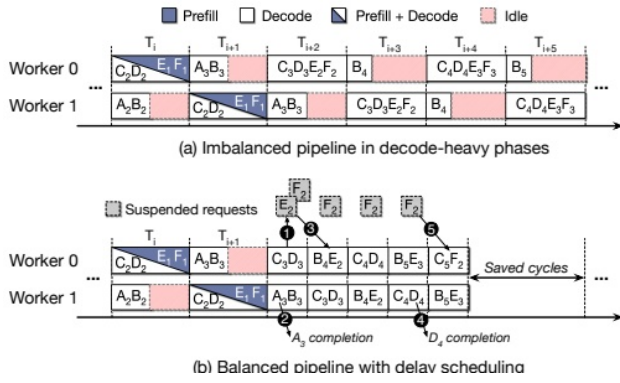


Figure 7: Reducing pipeline stalls with delay scheduling

图组解读 消融验证 呼应：第 6 节：模块三延迟调度 (DS)

- **图组结论：**DS 通过按 TPOT 延迟余量挂起和恢复 decode 请求，减少迭代间的空闲格。
- **论证关系：**为 DS 模块应对纯 decode (D-D) 不均衡的设计提供机制示意与逻辑支撑。
- **适用范围：**简化 Gantt 图示意，不包含具体延迟数值改善幅度。

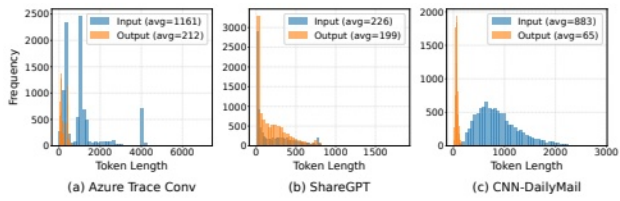


Figure 8: Input and output length distributions

图组解读 适用边界 呼应：第 8 节：实验负载特征与分类

- **图组结论：**AzureConv/CNN 呈 prefill-heavy 特征（输入均值远大于输出），ShareGPT 输入输出均值接近。
- **论证关系：**为实验负载分类及 CNN-Reversed 构造提供数据支撑。
- **适用范围：**三条真实 trace 统计，未单独包含 CNN-Reversed 分布图。

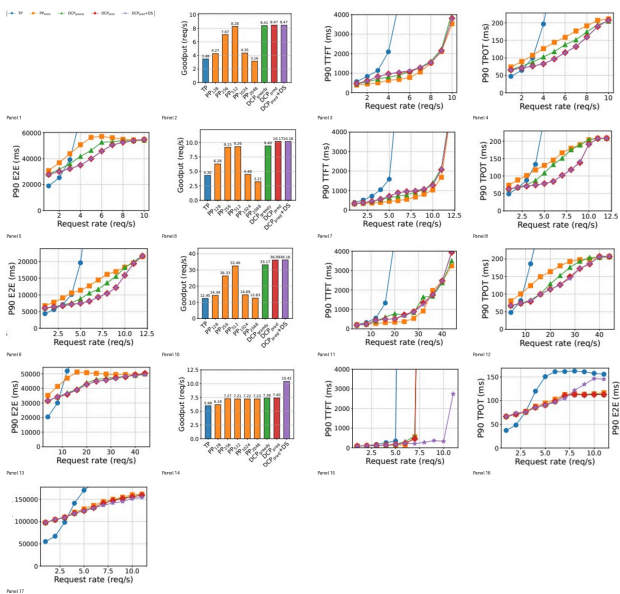


Figure 9: Goodput, TTFT, TPOT, and E2E latency for Qwen2.5-32B

图组解读 实验结论 呼应：第 8 节：32B 模型主实验结果

- **图组结论：**prefill-heavy 负载下 DCP 大幅压低 TPOT/E2E；CNN-Reversed 上仅 DCP_pred+DS 显著提升 goodput (7.27→10.42)。
- **论证关系：**直接支撑“调优 PP 超越 TP”及“DS 对 decode-heavy 不可或缺”核心主张。
- **适用范围：**Qwen2.5-32B 4×A100 PCIe 部署，个别子图存在标签重叠。

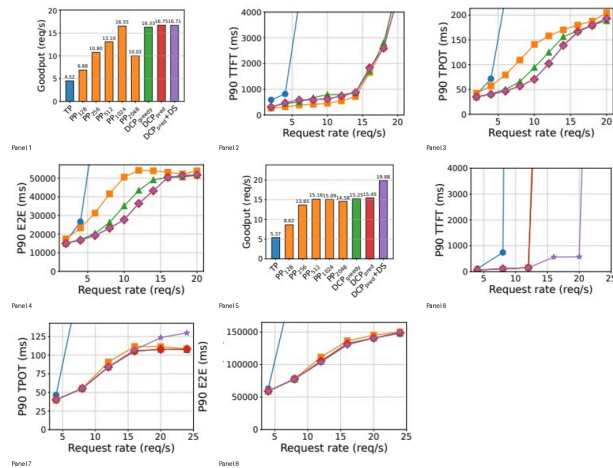


Figure 10: Goodput, TTFT, TPOT, and E2E latency for Qwen2.5-14B

图组解读 适用边界 呼应：第 8 节：14B 模型规模泛化性

- **图组结论：**14B 呈现类似 goodput 提升，但 DS 对 P90 E2E 延迟无可见改善。
- **论证关系：**验证核心结论跨模型规模的泛化性，同时暴露 E2E 收益的规模依赖边界。
- **适用范围：**覆盖 14B 模型在 AzureConv 与 CNN-Reversed 上的表现。

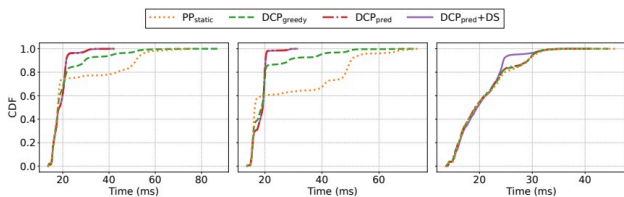


Figure 11: CDF of iteration latency for (a) AzureConv, (b) CNN, and (c) CNN-Reversed

图组解读 消融验证 呼应：第 8 节：迭代延迟 CDF 分析

- **图组结论：**AzureConv/CNN 上 DCP_pred 曲线左移最明显；CNN-Reversed 上仅 DCP_pred+DS 单独左移。
- **论证关系：**支撑 DCP 压缩 prefill 延迟、DS 是 decode-heavy 残余不均衡必要补充的机制主张。
- **适用范围：**多曲线高度重合，无法从图像像素读出精确数值。

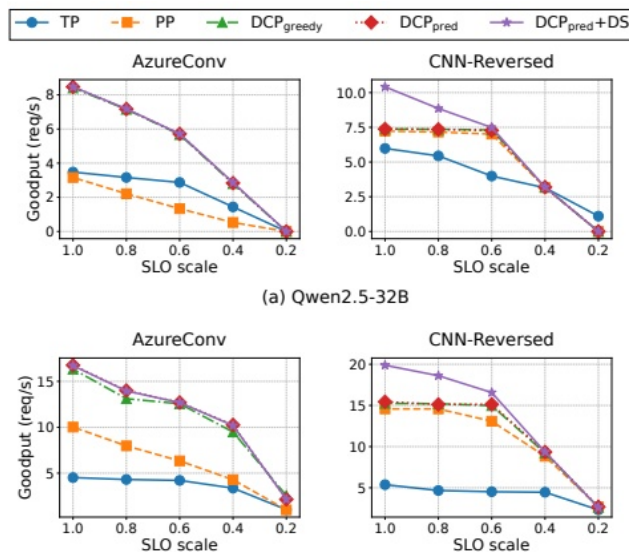


Figure 12: Goodput comparison of Qwen2.5 32B and 14B under various SLO constraints.

图组解读 适用边界 呼应：第 8 节：SLO 约束敏感性实验

- **图组结论：**DCP 相对 PP/TP 的 goodput 优势在 SLO 收紧时保持并增大；CNN-Reversed 下 DS 保持分离。
- **论证关系：**支撑“DCP 提升随 SLO 收紧增大”的稳健性与适用范围主张。
- **适用范围：**五档 SLO scale，部分曲线重叠需结合正文数值判读。

10. 完整因果推理树

- 根问题：PCIe 环境下 PP 能否在 SLO 约束下超越 TP？
 - ▶ 旧方法限制：TP 通信瓶颈；静态 chunked-prefill 与静态微批调度无法适应在线负载变化 **作者明确陈述**
 - ▶ 核心洞见/假设：气泡源于阶段间计算不均衡 (P-P/P-D/D-D)，在线场景加剧 **作者明确陈述**
 - 方法模块：DCP_greedy (反馈式调 chunk)
 - 可验证预测：动态调 chunk 可降低 TPOT/E2E 且不损 goodput
 - ▶ 指标与实验：5.2.1 节 TPOT/E2E 降幅 19%/18% (AzureConv) **实验支持**
 - 图表证据：Figure 9 曲线左移与压低
 - 结论与适用边界：适用于 prefill-heavy 负载；对纯 D-D 不均衡无效
 - 方法模块：DCP_pred (ALP 预测式调 chunk)
 - 可验证预测：预测式优于贪心式，尾延迟更短

- 指标与实验: Figure 11 CDF、Table 1 MAPE=3.26% **实验支持**
 - 图表证据: Figure 11 曲线爬升更陡
 - 结论与适用边界: 未验证跨模型/硬件的泛化性 [无法从当前 OCR 内容确认]
- 方法模块: Delay Scheduling (跨阶段请求重排)
 - 可验证预测: D-D 不均衡下 DS 显著提升 goodput, 但增加被抢占请求的 TPOT
 - 指标与实验: CNN-Reversed goodput 提升至 1.4×; P99 TPOT 变差 15% **实验支持**
 - 图表证据: Figure 9/12 中 DS 曲线跃升与右移
 - 结论与适用边界: 仅在 decode-heavy 场景激活; 对 14B 模型 P90 E2E 无可见改善

11. 结论、贡献与边界

贡献: 对在线 PP 不均衡的三类归因分析; 贪心式动态分块 (Algorithm 1); 预测式动态分块 (ALP+RLS); 延迟调度机制 **作者明确陈述**。

证据充分的结论: 在 4×A100 PCIe、Qwen2.5 32B/14B 上, DCP+DS 使 PP 在 goodput、TPOT/E2E 尾延迟上优于 TP 与静态 PP 基线 **实验支持**。

尚属推断的意义: 该方案对更大规模集群、NVLink 混合互联、MoE 模型的适用性 **[基于文中逻辑的推断, 未直接验证]**。

局限: 关键超参数与抢占阈值未公开取值; MAPE 等指标公式未给出显式表达式; 未与 [6]、phase-disaggregated serving 做直接实验对比; 仅评估两个模型规模、单一硬件配置 [文中未说明][无法从当前 OCR 内容确认]。

审稿式风险检查:

- 贡献新意: 三类不均衡归因与调度层设计为增量创新, 未涉及新并行范式, 当前证据未显示该风险之外的过度包装。
- 写作/可复现性: 关键超参数缺失取值, 复现存在障碍, 此风险由证据本身 (8.2 节“文中未说明”) 暴露。
- 效果大小: 多组实验数值一致支持效果, 但部分改善 (如 ShareGPT) 幅度小于其他负载, 需结合负载特性理解。
- 实验覆盖: 仅两模型、单一 GPU/互联配置, 覆盖面较窄, 属已暴露的适用范围缺口。
- 方法设计: DS 带来的 TPOT 代价 (P99 变差 15%) 是作者自陈的权衡而非风险掩盖, 当前证据未显示未披露的负面权衡。

12. 术语与第一性原理学习路径

12.1 核心术语

- **术语:** 流水线并行 (PP) —— 把模型按层切成若干段, 分别放在不同 GPU 上, 请求按微批次依次流过各段。 **第一性原理角色:** 受“单 GPU 显存容量”约束驱动, 用通信量换取显存可行性。 **本文位置:** 全文分析对象。
- **术语:** 流水线气泡 —— 某 GPU 因等待前一阶段完成而空闲的时间段。 **第一性原理角色:** 连接“阶段间计算量差异”与“资源利用率下降”两个量。 **本文位置:** 核心待解决问题。
- **术语:** Chunk size (分块大小) —— 一次迭代中处理的 prefill token 数量上限。 **第一性原理角色:** 直接决定线性层计算量 N_{tokens} , 是调度可控变量。 **本文位置:** DCP 模块的决策对象。
- **术语:** Goodput —— 同时满足 P90 TTFT 与 P90 TPOT SLO 的最大请求速率。 **第一性原理角色:** 把“延迟约束”转化为“可支持吞吐”这一单一可比较量。 **本文位置:** 核心评估指标。
- **术语:** RLS (递归最小二乘) —— 每来一个新数据点即更新参数估计的在线学习算法。 **第一性原理角色:** 连接“实测延迟”与“预测模型系数”, 无需重新训练整体模型。 **本文位置:** ALP 在线校准 β 系数。
- **术语:** Delay Scheduling (延迟调度) —— 按 TPOT 余量挂起/恢复微批中的部分 decode 请求。 **第一性原理角色:** 连接“请求数不均”与“硬件批次效率边界”两个量。 **本文位置:** 模块三, 专治 D-D 不均衡。

12.2 学习路径

1. LLM 推理的 prefill/decode 两阶段区别 —— 理解为何计算量会因阶段而异, 这是所有不均衡分析的起点。
2. KV Cache 的作用 —— 理解 decode 阶段为何计算量近似恒定, 从而理解 D-D 不均衡的成因。
3. 模型并行 (TP vs PP) 的通信/计算权衡 —— 理解为何 PCIe 环境天然偏好 PP。
4. SLO 与 goodput 的关系 —— 理解本文如何把“更快”转化为可比较的单一指标。
5. 在线反馈控制的基本思想 (贪心 vs 预测) —— 理解 DCP_greedy 与 DCP_pred 的设计差异根源。